Monte Carlo Tree Search and Hex: A Topological and Probabilistic Exploration

Demonstration Project Documentation

Abstract

This paper explores the mathematical foundations and practical implementation of Monte Carlo Tree Search (MCTS). We contextualize the algorithm through the game of Hex, a connection game with profound topological implications. By examining the historical development of both the game and the algorithm, we detail why deterministic search methods fail in high-complexity spaces and how MCTS overcomes these bounds via probabilistic sampling. We detail two distinct computational paradigms: classical MCTS backed by an $\mathcal{O}(\alpha(V))$ Union-Find explicit simulator, and zero-knowledge Latent MCTS (MuZero) operating within a learned neural manifold. Finally, we present the architecture of an interactive demonstration deployed as a client-side Progressive Web Application (PWA) with a WebSocket-enabled PyTorch backend.

1 Introduction

The challenge of navigating immense combinatorial spaces has historically driven advancements in applied mathematics and computer science. Traditional decision-making algorithms, such as Minimax augmented with α-β pruning, require the exploration of finite game trees. While highly successful in environments with manageable branching factors and reliable heuristic evaluation functions, these deterministic methods falter in domains characterized by massive state spaces or abstract positional values.

Monte Carlo Tree Search (MCTS) offers an elegant alternative by merging the precision of tree search with the statistical power of random sampling. Rather than exhaustively evaluating all possible futures, MCTS builds an asymmetric decision tree, focusing computational resources strictly on the most promising variations based on empirical results. To illustrate this algorithm in practice, we examine its application to the game of Hex.

2 The Game of Hex: Topology and Combinatorics

Invented independently by Danish mathematician Piet Hein in 1942 and by John Nash at Princeton University in 1948, Hex is a zero-sum connection game played on a rhombus grid

of hexagonal cells. Two players attempt to form an unbroken chain of connected stones between opposing sides of the board. In the context of our demonstration application, the Human player (Red) attempts a Top-to-Bottom connection, while the algorithmic agent (Blue) attempts a Left-to-Right connection.

2.1 Topology and the Brouwer Fixed-Point Theorem

Unlike many combinatorial games, a completed game of Hex can never end in a draw. This property is deeply rooted in topology. Gale (1979) demonstrated that the "Hex Theorem" (the impossibility of a draw) is mathematically equivalent to the two-dimensional Brouwer Fixed-Point Theorem. Thus, playing Hex is fundamentally an exercise in constructing and blocking continuous mappings across a topological space.

2.2 The Strategy-Stealing Argument and Computational Complexity

Nash famously proved that the first player has a guaranteed winning strategy on any symmetric $n \times n$ board (Gale, 1979; Gardner, 1959). His non-constructive proof utilizes a *strategy-stealing argument*: assume, for the sake of contradiction, that the second player has a winning strategy. The first player could place an arbitrary initial tile and then mentally adopt the second player's winning strategy. Because an extra tile on the board can never disadvantage a player in Hex (the game is strictly monotonic), the first player can force a win, contradicting the initial assumption.

While this proves a winning strategy exists, it provides no computational method for finding it. Reisch (1981) established that determining the winner of an arbitrary Hex position is PSPACE-complete. The state space complexity of an 11×11 board is approximately $\mathcal{O}(10^{43})$, rendering explicit calculation impossible and making Hex an ideal candidate for probabilistic search.

3 Monte Carlo Tree Search

3.1 Historical Context

The foundation of MCTS rests on the Monte Carlo method, formalized in the 1940s by Stanislaw Ulam and Nicholas Metropolis (Metropolis & Ulam, 1949). In game theory, Abramson (1990) explored using random rollouts to evaluate game tree leaves.

The modern instantiation of MCTS emerged in 2006. Coulom (2006) coined the term "Monte Carlo Tree Search," and independently, Kocsis and Szepesvári (2006) published the breakthrough UCT (Upper Confidence bounds applied to Trees) algorithm. UCT maps the multi-armed bandit problem onto tree traversal, allowing for asymmetric, statistically guided exploration. MCTS achieved global prominence as the core algorithm behind AlphaGo (Silver et al., 2016).

3.2 The Four-Phase Algorithm

MCTS navigates a state space modeled as a directed acyclic graph. It builds a search tree incrementally by iterating through four distinct phases until a computational budget is exhausted:

1. **Selection:** Starting from the root node, a tree policy navigates down to a leaf node. The selection balances *exploitation* (choosing nodes with a high empirical win rate) and *exploration* (choosing rarely visited nodes). This is governed by the UCB1 formula:
$$\mathrm{UCT}_i = \frac{w_i}{n_i} + c\sqrt{\frac{\ln N_i}{n_i}} \tag{1}$$
where w_i is the number of wins for child node i, n_i is its visit count, N_i is the parent node's visit count, and c is the exploration constant.

2. **Expansion:** If the selected leaf node is not a terminal state, the tree is expanded by generating valid child nodes representing legal moves.

3. **Simulation (Rollout):** From a newly expanded node, a simulation is run to a terminal state using a random rollout policy.

4. **Backpropagation:** The terminal outcome is propagated backward up the traversed path, updating n_i and w_i for each node.

Algorithm 1 Classical Explicit MCTS (Disjoint-Set Rollout Engine)

function ClassicalMCTS(s_0, K)
  Initialize root node R with state s_0
  for $k \leftarrow 1$ to K **do**
    $node \leftarrow R$, $state \leftarrow s_0$
    while *node* is fully expanded and not terminal **do** ▷ 1. Selection
      $action \leftarrow \arg\max_a \left[\frac{w(a)}{n(a)} + c\sqrt{\frac{\ln N(node)}{n(a)}}\right]$
      $state \leftarrow \text{ApplyAction}(state, action)$
      $node \leftarrow node.children[action]$
    end while
    if *node* is not terminal **then** ▷ 2. Expansion
      $action \leftarrow \text{UnvisitedAction}(node)$
      $state \leftarrow \text{ApplyAction}(state, action)$
      $node \leftarrow \text{AddChild}(node, action)$
    end if
    $sim\_state \leftarrow \text{Copy}(state)$ ▷ 3. Simulation
    while not UnionFindIsTerminal(sim_state) **do**
      $a_{rand} \leftarrow \text{RandomLegalMove}(sim\_state)$
      $sim\_state \leftarrow \text{ApplyAction}(sim\_state, a_{rand})$
    end while
    $reward \leftarrow \text{EvaluateOutcome}(sim\_state)$
    while $node \neq$ Null **do** ▷ 4. Backpropagation
      $node.n \leftarrow node.n + 1$
      $node.w \leftarrow node.w + reward$
      $reward \leftarrow -reward$ ▷ Alternating Minimax Perspective
      $node \leftarrow node.parent$
    end while
  end for
  return $\arg\max_a n(a)$
end function

3.3 Zero-Knowledge Variants: Latent MCTS (The MuZero Paradigm)

Classical MCTS requires an explicit environment simulator (the game rulebook) to simulate rollouts. In contrast, modern zero-knowledge algorithms such as MuZero (Schrittwieser et al., 2020) operate entirely within a learned latent state space.

MuZero decomposes the environment into three deep PyTorch neural networks parameterized by θ:

1. **Representation Network (h_θ):** Embeds historical observations $o_1, \ldots, o_t$ into an initial spatial latent state s_0:

$$s_0 = h_\theta(o_1, \ldots, o_t) \tag{2}$$

2. **Dynamics Network (g_θ):** Predicts the next latent state s_k and intermediate reward r_k given a current latent state s_{k-1} and candidate action a_k:

$$r_k, s_k = g_\theta(s_{k-1}, a_k) \tag{3}$$

3. **Prediction Network (f_θ):** Computes policy priors $\mathbf{p}_k$ and scalar value estimate $v_k \in [-1, 1]$ directly from a latent state s_k:

$$\mathbf{p}_k, v_k = f_\theta(s_k) \tag{4}$$

During search, Latent MCTS selects actions using the Predictor Upper Confidence Bound (PUCT) formula:

$$\text{PUCT}(s, a) = Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)} \tag{5}$$

where $P(s, a)$ is supplied by the prediction head f_θ. Because search occurs entirely inside the continuous manifold defined by h_θ and g_θ, the algorithm plans effectively without querying environment rules or performing physical rollouts.

Algorithm 2 Latent MCTS (MuZero Latent Space Paradigm)

function LATENTMCTS(o_{raw}, K)
  $s_0 \leftarrow h_\theta(o_{raw})$ ▷ 1. Representation Embedding
  $\mathbf{p}_0, v_0 \leftarrow f_\theta(s_0)$
  Initialize root R with state s_0 and priors $\mathbf{p}_0$
  for $k \leftarrow 1$ to K **do**
    $node \leftarrow R$, $path \leftarrow []$
    while $node$ is expanded **do** ▷ 2. Latent Selection
      $action \leftarrow \arg\max_a \left[Q(node, a) + c_{\text{puct}} P(node, a) \frac{\sqrt{N(node)}}{1+N(node,a)} \right]$
      Append $(node, action)$ to $path$
      $node \leftarrow node.children[action]$
    end while
    $(parent, a_{last}) \leftarrow path[-1]$ ▷ 3. Recurrent Dynamics Expansion
    $r_{leaf}, s_{leaf} \leftarrow g_\theta(parent.hidden\_state, a_{last})$
    $\mathbf{p}_{leaf}, v_{leaf} \leftarrow f_\theta(s_{leaf})$
    $node.hidden\_state \leftarrow s_{leaf}$
    Expand $node.children$ with priors $\mathbf{p}_{leaf}$
    $val \leftarrow v_{leaf}$
    for each (n, a) in Reversed($path$) **do** ▷ 4. Alternating Backpropagation
      $n.children[a].visit\_count \leftarrow n.children[a].visit\_count + 1$
      $n.children[a].value\_sum \leftarrow n.children[a].value\_sum + val$
      $val \leftarrow -val$ ▷ Adversarial Zero-Sum Inversion
    end for
  end for
  return $\arg\max_a N(R, a)$
end function

4 Neural Optimization and Empirical Benchmarking

4.1 Backpropagation Through Time (BPTT) and Dynamics Optimization

Training the MuZero dynamics model g_θ requires unrolling network predictions across sequence steps using Backpropagation Through Time (BPTT). Given a trajectory of observations o_t, target policies π_t, and terminal values z_t, the loss function $\mathcal{L}(\theta)$ is evaluated across $K = 5$ unrolled steps:

$$\mathcal{L}(\theta) = \sum_{k=0}^{K} \left(l_p(\mathbf{p}_{t+k}, \hat{\mathbf{p}}_{t+k}) + l_v(v_{t+k}, \hat{v}_{t+k}) \right) \tag{6}$$

where l_p represents cross-entropy policy loss and l_v denotes mean squared error value loss. To prevent gradient explosion through deep unrolled hidden states, gradient scaling hooks are attached to the latent tensor:

$$\nabla_{s_k}\mathcal{L} \leftarrow 0.5 \times \nabla_{s_k}\mathcal{L} \tag{7}$$

Furthermore, loss evaluation masks out padded terminal steps to prevent target distortion.

4.2 Statistical Hypothesis Verification via SPRT

To rigorously confirm that the trained Latent MCTS agent performs significantly better than random chance, we employ Wald's Sequential Probability Ratio Test (SPRT). Pitting MuZero against a 1-simulation Classic MCTS opponent (a uniform random player), we compute the running log-likelihood ratio (LLR):

$$\text{LLR}_m = \sum_{i=1}^{m} \ln \frac{P(y_i \mid H_1)}{P(y_i \mid H_0)} \tag{8}$$

where $H_0 : p \leq 0.50$ and $H_1 : p \geq 0.55$. The test terminates as soon as LLR crosses upper bound A or lower bound B:

$$A = \ln\left(\frac{1-\beta}{\alpha}\right), \quad B = \ln\left(\frac{\beta}{1-\alpha}\right) \tag{9}$$

Setting Type I error $\alpha = 0.05$ and Type II error $\beta = 0.05$ yields bounds $A \approx 2.944$ and $B \approx -2.944$, providing optimal decision efficiency.

4.3 Classic Simulation Equivalent (CSE) Estimation via Logistic Regression

To quantify neural compute efficiency without the instability of standard binary search, we estimate the Classic Simulation Equivalent (CSE) using 4-parameter logistic regression. We measure MuZero's win rate against Classic MCTS across a logarithmic grid of anchor simulation counts $X \in \{1, 5, 10, 25, 50, 100, 200, 400\}$. We fit a Sigmoid model:

$$P(\text{win}) = \frac{1}{1 + e^{-k(\ln(X) - \ln(\text{CSE}))}} \tag{10}$$

where k governs the win-rate slope and CSE represents the exact simulation count at parity ($P(\text{win}) = 0.50$). Global non-linear regression across all anchor points averages out single-match noise, producing a robust, highly reproducible evaluation metric.

5 Broader Applications of MCTS

Because MCTS is a generic decision-making framework capable of navigating arbitrary combinatorial spaces, its applications extend far beyond zero-sum games:

- **Combinatorial Optimization:** DeepMind's AlphaTensor (Fawzi et al., 2022) framed matrix multiplication as a single-player game, using MCTS to discover novel tensor decomposition algorithms that outperform long-standing human-designed baselines.
- **Automated Theorem Proving:** AI systems represent geometric axioms as starting states and logical deductions as actions, using MCTS to search the space of valid proofs for olympiad-level mathematics.
- **Generative AI:** Advanced reasoning models employ inference-time compute algorithms inspired by MCTS to navigate "trees of thought," systematically verifying intermediate logical steps before outputting final answers.

6 Implementation in the Interactive Demonstration

The accompanying demonstration project[1] operates across two distinct mathematical regimes: an offline client-side Progressive Web Application (PWA) executing explicit Union-Find simulations, and a WebSocket-connected Python backend running deep PyTorch Latent MCTS.

6.1 Numerical Bounding of the Combinatorial Explosion

A central feature of the application is a "Cosmic Inspection Panel" that dynamically calculates $E!$, the number of possible future playout sequences (where E is the number of empty cells).

On an 11×11 board, calculating $E!$ trivially exceeds the maximum representable value of IEEE 754 double-precision floating-point format (1.79×10^{308}). To prevent numeric overflow (which yields `Infinity` in standard web environments), the application computes the permutations using a summation of base-10 logarithms:

$$\log_{10}(E!) = \sum_{i=1}^{E} \log_{10}(i) \tag{11}$$

The application extracts the exponent as $X = \lfloor \sum \log_{10}(i) \rfloor$ and the mantissa as $M = 10^{\sum \log_{10}(i) - X}$. This allows the UI to safely format the vast state space (e.g., 1.24×10^{61}) and map it against cosmic scale thresholds.

[1] Live application available at https://tryggth.github.io/hex/, with source code hosted at https://github.com/tryggth/hex.

6.2 Efficient Win Detection via Disjoint-Set Data Structures

In the offline explicit simulator regime, terminal win detection must be exceptionally fast. Re-evaluating path connectivity using Breadth-First Search at every move is computationally prohibitive. The application addresses this by implementing a Disjoint-Set (Union-Find) data structure with path compression and union by rank (Tarjan, 1975). Adjacent cells are merged into connected components in near-constant amortized time $\mathcal{O}(\alpha(V))$, allowing instant verification of top-to-bottom or left-to-right connectivity.

6.3 Non-Linear Scaling of the Attention Heatmap

During real-time strategy generation, the application visualizes search density by modulating the internal fill color of the hexagonal cells on the HTML5 canvas. Because MCTS visit counts (v) form a heavily skewed distribution, a linear color mapping would obscure secondary candidate moves. To resolve this, the interface applies a fractional power curve mapped to the normalized visit count:

$$t = \left(\frac{v}{v_{\max}} \right)^{0.5} \tag{12}$$

Applying the square root artificially inflates the visual weight of mid-tier candidates, allowing the observer to clearly track the algorithm's shifting attention as it prunes unviable subtrees in real time.

6.4 The Anytime Algorithm Property and Parameterization

Because MCTS operates iteratively rather than deterministically, it possesses the "anytime" algorithm property—meaning it can be halted at any arbitrary millisecond and still return a mathematically rigorous best guess based on the asymmetrical tree built thus far. The UI actively demonstrates this theorem via a "Stop Strategizing" interrupt feature. By manually halting the execution thread, the application ceases iterations, evaluates the current root tree, selects the candidate node with the highest visit count, and immediately yields control back to the human player.

Furthermore, users can manipulate the exploration parameter c from Equation (1). By dynamically adjusting this value via a UI slider, observers can perturb the algorithm. Setting $c \approx 0$ forces a greedy policy prone to local optima, while high values induce undirected noise, illustrating empirically why default exploration constants maintain theoretically optimal convergence.

7 Conclusion

Monte Carlo Tree Search represents a paradigm shift in navigating combinatorial complexity. By substituting exhaustive calculation with probabilistically bounded sampling, MCTS effectively deduces mathematically robust strategies without relying on

pre-computed evaluation tables. The dual-mode Hex application serves as a tangible manifestation of these principles, contrasting an explicit $\mathcal{O}(\alpha(V))$ Union-Find engine with a continuous neural latent manifold.

References

Abramson, B. (1990). Expected-outcome single-agent search. *Computational Intelligence*, *6*(2), 98–105. https://doi.org/10.1111/j.1467-8640.1990.tb00135.x

Coulom, R. (2006). Efficient selectivity and backup operators in Monte-Carlo tree search. In *International Conference on Computers and Games* (pp. 72–83). Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-75538-8_7

Fawzi, A., Balog, M., Huang, A., Hubert, T., Romera-Paredes, B., Barekatain, M., Novikov, A., Rusu, A. A., Ruiz, F. J., Pozdnyakov, A., Woods, G., Prokopec, A., Richard, R., Yang, R., Bamiec, M., Balle, M., Hassabis, D., & Kohli, P. (2022). Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, *610*(7930), 47–53. https://doi.org/10.1038/s41586-022-05172-4

Gale, D. (1979). The Game of Hex and the Brouwer Fixed-Point Theorem. *The American Mathematical Monthly*, *86*(10), 818–827. https://doi.org/10.1080/00029890.1979.11994922

Gardner, M. (1959). *The Scientific American Book of Mathematical Puzzles and Diversions*. Simon and Schuster.

Kocsis, L., & Szepesvári, C. (2006). Bandit based Monte-Carlo planning. In *European Conference on Machine Learning* (pp. 282–293). Springer, Berlin, Heidelberg. https://doi.org/10.1007/11871842_29

Metropolis, N., & Ulam, S. (1949). The Monte Carlo method. *Journal of the American Statistical Association*, *44*(247), 335–341. https://doi.org/10.1080/01621459.1949.10483310

Reisch, S. (1981). Hex ist PSPACE-vollständig. *Acta Informatica*, *15*(2), 167–191. https://doi.org/10.1007/BF00288964

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., Lillicrap, T., & Silver, D. (2020). Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, *588*(7839), 604–609. https://doi.org/10.1038/s41586-020-03051-4

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, *529*(7587), 484–489. https://doi.org/10.1038/nature16961

Tarjan, R. E. (1975). Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, *22*(2), 215–225. https://doi.org/10.1145/321879.321884